

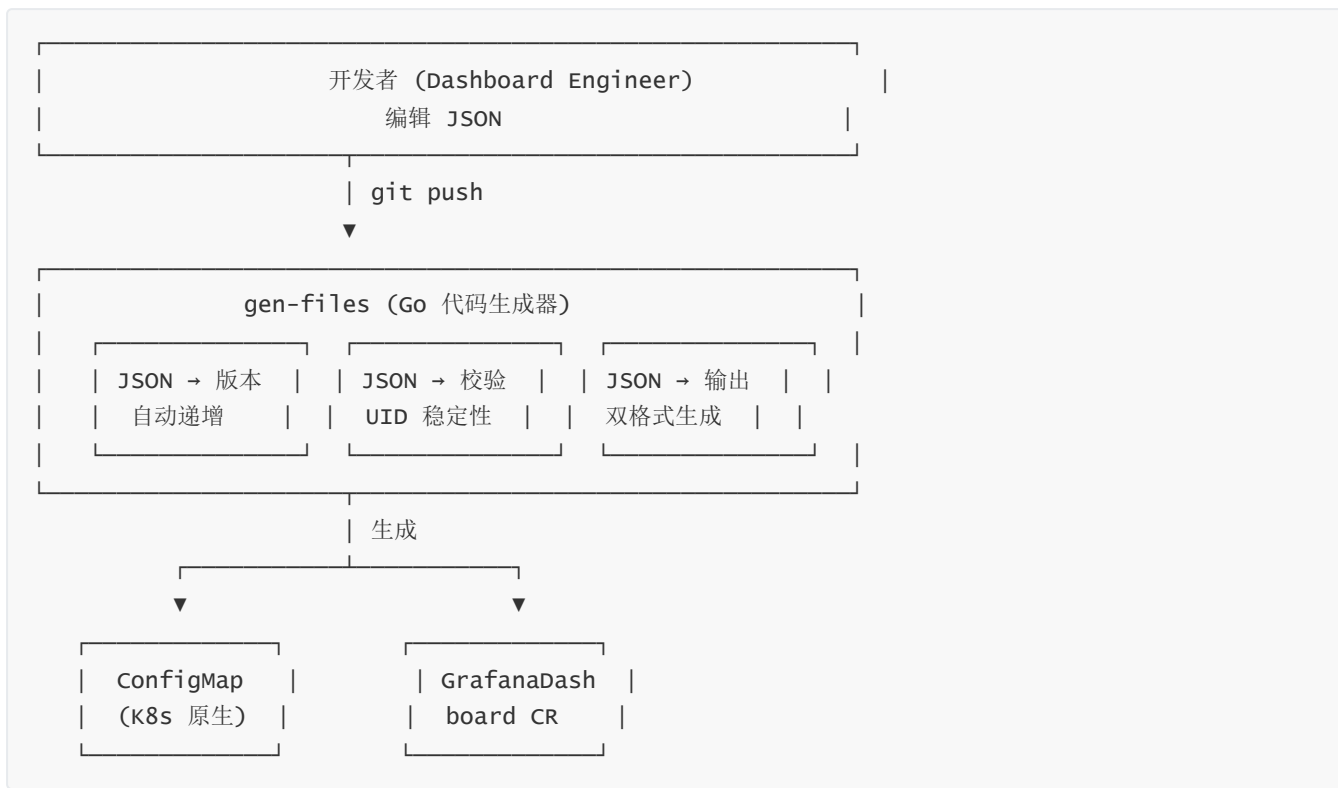
监控平台技术面试文档

项目概述

负责 Casa Systems 5G 核心网网元 (SMF、OAM、Platform) 的可观测性平台建设，涵盖 **Grafana 仪表盘设计**、**Prometheus 告警体系**、**自动化工具链** 和 **CI/CD 流水线**，支撑 12 个网络功能应用的监控资源管理。

一、架构设计

1.1 整体架构：三层分离 + 单一数据源



核心理念：单一数据源 (Single Source of Truth)

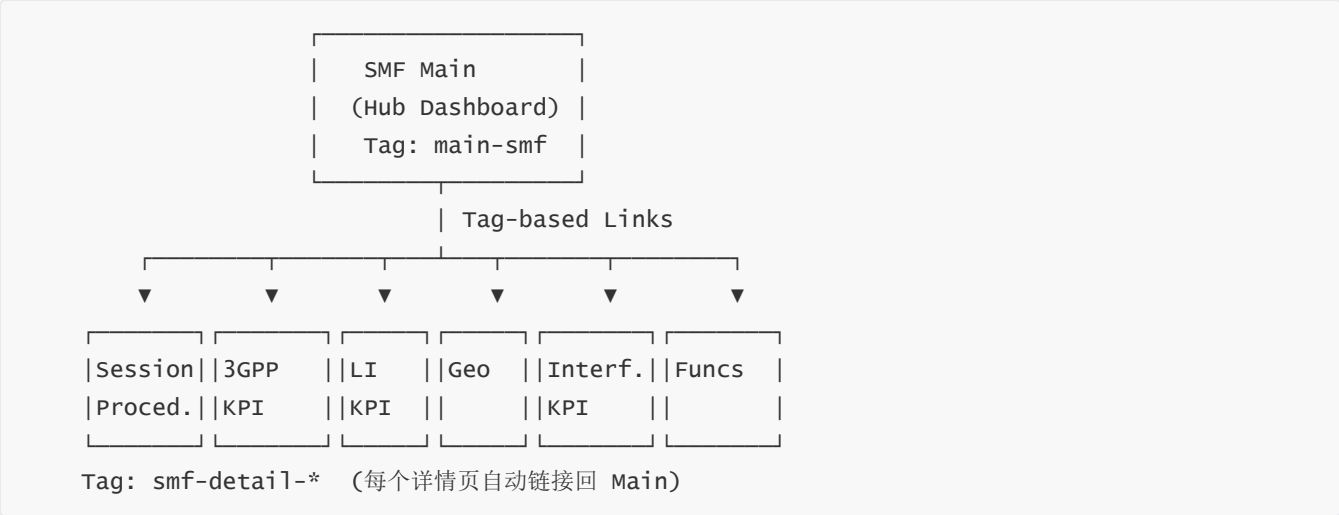
文件类型	存储位置	编辑方式	说明
源 JSON	json/	手动编辑	唯一人工维护的文件
ConfigMap	configmaps/	自动生成	K8s ConfigMap, 嵌入完整 JSON
GrafanaDashboard CR	grafana-operator/	自动生成	Grafana Operator 自定义资源
PrometheusRule	prometheus-rules/	手动编辑	告警规则 YAML

技术亮点：

- 零手工重复**：开发者只编辑一个 JSON 文件，生成器自动产出两个 K8s 资源
- Git-diff 驱动**：gen-files 通过 git diff 精准识别变更文件，只处理差异部分

- CI 强制一致性: `verify-resources` job 在每次提交时重新生成并比对, 任何不一致直接阻断流水线

1.2 仪表盘导航架构: Hub-and-Spoke 模型



设计亮点:

- 基于 Grafana 标签的动态链接, 无需硬编码仪表盘 URL
- Main 仪表盘作为导航中心, 提供全局视角 (UE 数、会话数、IP 池、连接状态)
- 详情仪表盘按功能域拆分 (Session Procedure、3GPP KPI、LI、GEO、Interface、Functions)
- 支持多维度下钻: 从全局概览 → 网元状态 → 接口级消息 → 函数级内部状态

二、SMF 核心技术亮点

2.1 多维度告警体系: 4 级严重度 + 双层告警

Success Rate 告警模型 (4 级递进) :

严重度	阈值	含义
Warning	99%	轻微波动, 需关注
Minor	98%	性能下降, 需排查
Major	97%	严重异常, 需立即处理
Critical	95%	服务不可用, 需紧急恢复

覆盖范围: 50+ 告警规则, 涵盖 7 大维度:

- 会话管理: PDU Session 建立/更新/删除成功率
- 协议接口: PCF、Gx、Gy、S6b、Rf Diameter 消息成功率
- 3GPP 服务: Nudm、Npcf、Nchf 等 NF 间通信成功率
- 基础设施: N4、S5/S8、S2b、Gn/Gp 路径连通性

- 5. **GEO 高可用**: Active/Standby 故障切换、BGP 状态
- 6. **资源使用**: IP 池使用率 (60/70/80/90% 四级)
- 7. **服务健康**: DB 断连、gRPC/HTTP/UDP 异常、VTAP 探测

双层告警设计:

Aggregate Alert (axnf-level)
例: BGP 全部 Down

→ Critical: 整体不可用

Pod-level Alert
例: 单个 Pod BGP Down

→ Minor: 单 Pod 故障

技术亮点:

- 操作员可快速区分"全局故障"vs"部分 Pod 故障"
- 每条告警携带 OID 标签 (1.3.6.1.4.1.20858.10.104.20.2.5.*), 对接 SNMP 网管系统
- 10 分钟 for 持续时间, 避免瞬时抖动误报

2.2 Recording Rules 预计算引擎

```
# 原始指标: 每个 SMF 实例单独上报
smf_sess_est_msg_counters{namespace="xxx", axnf="smf-0", code="2.00"}

# Recording Rule 聚合后:
job:smf_sm_counter:sum = sum by(namespace, axnf)(rate(smf*_msg_counters[5m]))
```

4 个预计算规则组:

规则组	用途	优化效果
smf_tps_aggregation_rule	消息吞吐量预聚合	避免仪表盘查询时实时聚合 10+ 接口指标
smf_db_service	数据库延迟直方图	smfsm/gtpc/n4/upfmgr/timermgr 5 个组件的 DB 延迟预计算
smf_timer_service	定时器回调延迟	各组件定时器启动/回调/停止延迟
smf_msg_service	消息处理全链路延迟	完整延迟 = 内部处理 + DB 操作, 支持拆分分析

技术亮点:

- 使用 label_replace + rate + sum 组合实现多维聚合
- 将 O(n²) 的仪表盘查询降为 O(n), 大幅降低 Prometheus 查询负载
- 支持"排除 DB 延迟"的变体, 精确定位应用层瓶颈

2.3 复杂 PromQL 表达式示例

```
# 多接口成功率聚合
sum(rate(smf_sm_grpc_pfcp_msg_counters{code="2.00"}[5m]))
/
sum(rate(smf_sm_grpc_pfcp_msg_counters[5m])) * 100

# IP 池使用率（带多级阈值）
smf_ip_pool_usage{namespace="$namespace"} / smf_ip_pool_total{namespace="$namespace"} * 100

# GEO 故障切换检测
increase(li_oam_num_localha_switchover[5m]) > 0
```

三、OAM 合规监控设计

3.1 LI-OAM 执法拦截监控体系

业务背景： LI (Lawful Interception) 是电信运营商的法律合规要求，OAM 系统负责管理 LI 订阅用户、NF 连接、审计日志和备份恢复。

6 个监控维度：

维度	关键指标	监控目标
用户状态	li_oam_users{state="active/inactive/locked"}	订阅用户生命周期
NF 连接	li_oam_nf_status{nf_type="SMF/UPF"}	与核心网网元的连通性
审计日志	li_oam_record_audit_logs_counters	合规审计追踪
备份恢复	li_oam_num_created/restored_oam_backups	数据保护能力
高可用	li_oam_num_localha_switchover	Pod 间故障切换
告警	li_oam_alarm_backup_fail	备份失败检测

NF 通信监控模式：



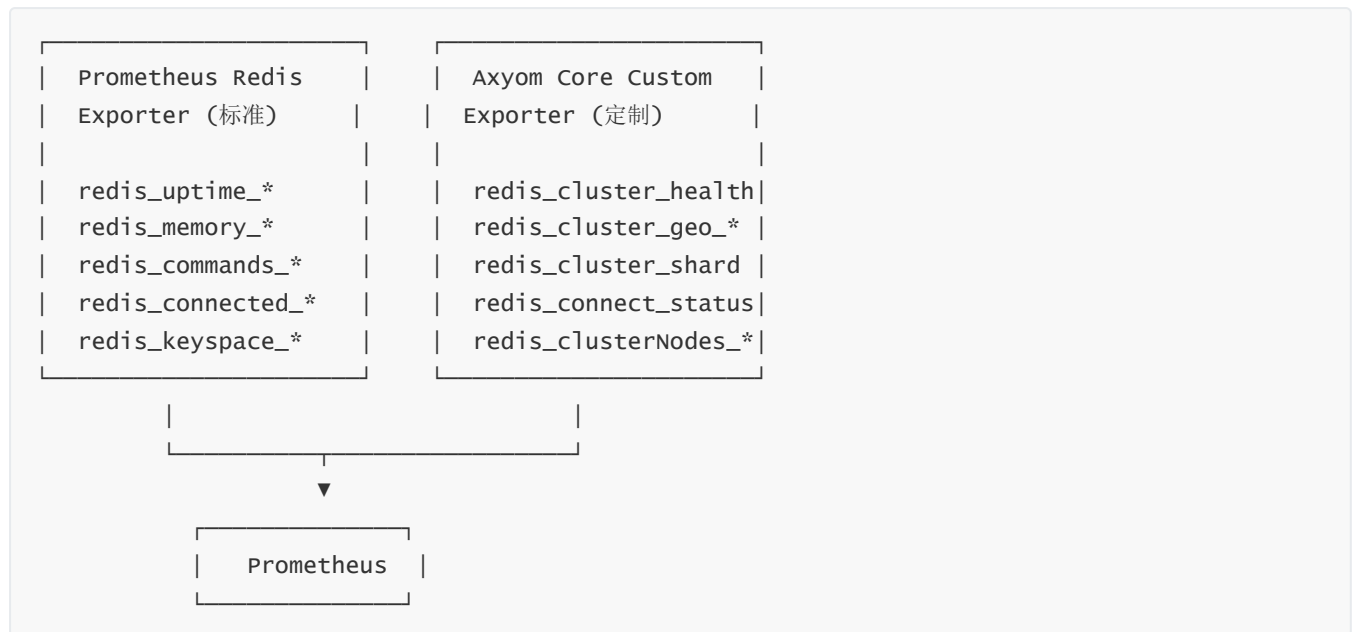
技术亮点：

- 按 HTTP 方法 + API 路径 + 响应码 + 原因的多维拆分

- 支持按 `nf_id` 精确定位故障 NF 实例
- Recording Rule 预聚合 + Alert Rule 两步告警模式

四、Platform Redis 集群监控

4.1 双 Exporter 架构



标准 Exporter 提供： 内存、CPU、命令吞吐、键空间、客户端连接

定制 Exporter 提供： GEO 拓扑状态、分片 HA、Leader 选举、同步健康

4.2 GEO 感知 Redis 集群监控

```
{
  "GEO States": ["Active", "Standby", "LocalOnly"],
  "Events": {
    "geo_switch_active": "主备切换",
    "geo_switch_standby": "备节点切换",
    "geo_switch_localonly": "本地模式切换",
    "shard_local_ha": "分片本地高可用",
    "shard_no_leader": "分片无主节点"
  }
}
```

告警覆盖： 18 条规则覆盖连接率、CPU、内存、GEO 切换、分片健康、命令失败等维度

4.3 Leader-Follower 复制健康监控

```
Leader ←—— Replication ———> Follower

|
| redis_master_link_up      |
| redis_master_repl_offset  |
| redis_slave_repl_offset   |
| redis_master_sync_in_progress |
| redis_replica_resyncs_full  |
| redis_replica_partial_resync |
| _accepted / _denied        |
|_____|
```

五、Go 自动化工具链

5.1 gen-files 核心算法

```
// 版本决策逻辑
func DetermineSemver(local, reference Version) Version {
    if local.Equal(reference) {
        return local.IncrementPatch() // 自动递增 patch
    }
    if reference.GreaterThan(local) {
        return reference.IncrementPatch() // 跟随 reference + patch
    }
    return local // 本地已是最新
}
```

关键设计决策：

- **UID 稳定性保障**：UID 变更 → 构建失败（防止仪表盘链接断裂）
- **JSON 字段保持**：自定义 `DashboardJSON` 结构体，通过 `json.RawMessage` 保留未知字段
- **字符转义修复**：自动还原 `\u0026` → `&`、`\u003e` → `>`、`\u003c` → `<`
- **Release 分支保护**：`release-*` 分支禁止 minor/major 版本升级

5.2 catalog 跨分支版本冲突防护

```
Master: smf v1.2.7, oam v1.0.2, platform v1.2.1 （上传到 GitLab Registry）
      |
      ▼
Release: smf v1.1.3 ← 比较 → v1.2.7 → 拒绝（版本落后）
      (release-9.M5)
```

机制：

1. Master 分支每次提交上传 `catalog.json` 到 GitLab Package Registry
2. Release 分支构建时下载 Master 的 catalog

- 3. 校验本地每个资源版本是否至少比 Master 高一个 minor
- 4. 不满足 → 构建失败，强制开发者在 Master 上先做版本升级

5.3 双注册表发布

```
Tag Push: applications/smf/v3.2.2
|
|─▶ GitLab Package Registry (CI_JOB_TOKEN)
|   └─ smf-monitoring-resources-v3.2.2.tgz
|
|─▶ JFrog Artifactory (ci-robot credentials)
|   └─ smf-monitoring-resources-v3.2.2.tgz
|   └─ 防重复上传检查
```

六、CI/CD 流水线设计

6.1 五层验证体系

Stage 1: catalog

版本冲突检测（跨分支安全网）

Stage 2: verify-resources

重新生成 → 比对 → 不一致则阻断

Stage 3: validate-json + validate-yaml

jq / yq 语法校验

Stage 4: reject-binaries

禁止二进制文件提交

Stage 5: validate-tag-and-push-package（仅 tag）

Tag 格式校验 → 打包 → 双注册表发布

6.2 版本管理策略

规则	实现
版本唯一来源	<code>annotations.list[0].version</code> (JSON 内部)
自动 Patch 递增	<code>gen-files</code> 检测到变更时自动 +1
Minor/Major 手动控制	开发者在 JSON 中修改后运行 <code>gen-files</code>
Release 分支保护	<code>release-*</code> 仅允许 Patch 升级
跨分支冲突检测	<code>catalog</code> 工具对比 Master 注册表
UID 不可变	变更 UID → 构建失败

七、技术数据汇总

7.1 项目规模

指标	数值
管理网元数	12 个应用 (agf, amf, mme, n3iwf, nrf, oam, platform, saeu, scp, sgw, smf, upf)
SMF 仪表盘数	10 个 (1 个主仪表盘 + 9 个详情仪表盘)
SMF 告警规则	50+ 条 (覆盖 7 大维度)
SMF Recording Rules	4 个规则组 (TPS、DB、Timer、Message)
OAM 仪表盘	1 个 (6 个监控维度)
OAM 告警规则	1 条 Critical (备份失败)
Platform 仪表盘	2 个 (单实例 + 多节点集群视图)
Platform 告警规则	18 条 (连接率、CPU、内存、GEO、分片)
Go 工具	5 个 (gen-files、catalog、version-bump、set-tags、extract.sh)

7.2 技术栈

层级	技术
可视化	Grafana (Schema v39-41)
指标采集	Prometheus + Prometheus Exporter
告警管理	PrometheusRule (monitoring.coreos.com/v1)
资源管理	Kubernetes ConfigMap + GrafanaDashboard CR

层级	技术
CI/CD	GitLab CI + Go 工具链
包管理	GitLab Package Registry + JFrog Artifactory
版本控制	Semantic Versioning (semver library)
语言	Go 1.23 (workspace 模式)

7.3 PromQL 技巧

```
# 1. 多接口成功率聚合
sum(rate(smf_sm_grpc_pfcg_msg_counters{code="2.00"}[5m]))
/ sum(rate(smf_sm_grpc_pfcg_msg_counters[5m])) * 100

# 2. IP 池使用率多级阈值
smf_ip_pool_usage / smf_ip_pool_total * 100 > 90 # critical

# 3. Recording Rule 预聚合
job:smf_sm_counter:sum = sum by(namespace, axnf)(
    rate(smf_sess_est_msg_counters[5m])
)

# 4. GEO 故障切换检测
increase(redis_cluster_geo_switch_active[5m]) > 0

# 5. Redis 连接率预计算
rediscluster:redis_connect_rate =
    redis_connected_clients / redis_config_maxclients * 100

# 6. 双维度告警（全局 + Pod 级）
# 全部 Down → Critical
BGP_Down_All = count(BGP_State == 0) == count(BGP_Total)
# 单 Pod Down → Minor
BGP_Down_Pod = BGP_State == 0 and BGP_Total > 0
```

八、面试亮点总结

8.1 架构能力

- 设计了 **单一数据源 + 代码生成** 的监控资源管理架构，消除手工重复
- 实现了 **Hub-and-Spoke 仪表盘导航**，支持 12 个网元的多维度下钻
- 构建了 **双 Exporter 架构**，整合标准与定制指标

8.2 工程能力

- 开发了 **Git-diff 驱动的代码生成器**，精准识别变更文件
- 实现了 **跨分支版本冲突检测**，防止 Release 分支与 Master 版本碰撞
- 设计了 **5 层 CI 验证体系**，从语法校验到一致性检查全覆盖

8.3 运维能力

- 设计了 **4 级严重度 + 双层告警** 模型，覆盖 50+ 告警规则
- 实现了 **Recording Rule 预计算引擎**，优化大规模指标查询性能
- 构建了 **SNMP OID 对接**，打通监控与传统网管系统

8.4 电信领域知识

- 深度理解 **3GPP TS 28.552** 性能指标标准
- 掌握 **SMF 会话管理**、**LI 执法拦截**、**GEO 高可用** 等核心电信概念
- 熟悉 **5G 核心网架构**：NRF、Nudm、Npcf、Nchf、PCF、GTPC 等接口

8.5 量化成果

成果	量化
管理监控资源	12 个应用，30+ 仪表盘，100+ 告警规则
自动化覆盖率	100%（所有 ConfigMap/CR 自动生成）
CI 验证层数	5 层（catalog、verify、JSON/YAML 校验、二进制检测）
告警响应时间	10 分钟 for 持续时间，避免误报
版本冲突防护	跨分支实时检测，阻止不兼容版本发布